PYTHON CONCURRENCY

Python без блокувань

Як працюють потоки?

Глибоке занурення у механізми синхронізації, Lock, черги та стратегії роботи з GIL.

Сумець Світлана

2026

Навіщо нам потоки?

> Потоки в Python є незамінними для задач, де процесор змушений чекати на зовнішні події, дозволяючи програмі залишатися ефективною.

I/O-Bound Задачі

Мережеві запити, робота з базою даних, читання файлів. Поки один потік чекає відповіді, інші працюють (GIL звільняється).

Фонова обробка

Виконання періодичних завдань (відправка пошти, логування, очищення кешу) без блокування основного циклу програми.

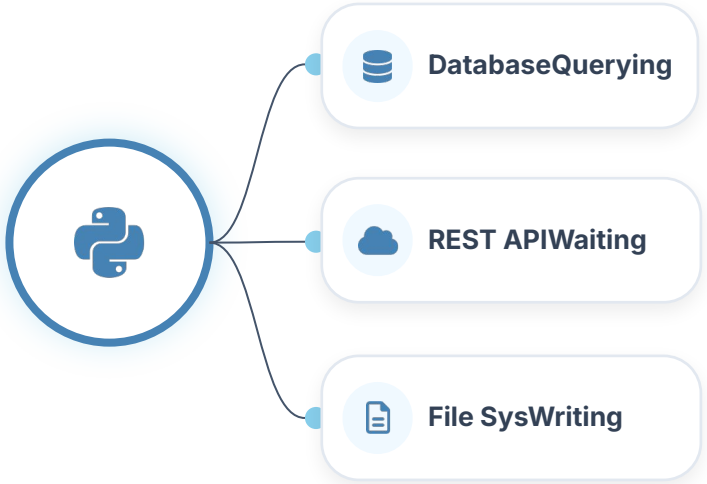
Реактивні інтерфейси

Запобігання "зависанню" GUI або веб-сервера під час тривалих операцій. Користувач отримує миттєвий відгук.

Обмеження GIL

Важливо пам'ятати: для важких математичних обчислень (CPU-bound) потоки не дають приросту швидкості через GIL.

КОНКУРЕНТНІСТЬ В ДІЇ

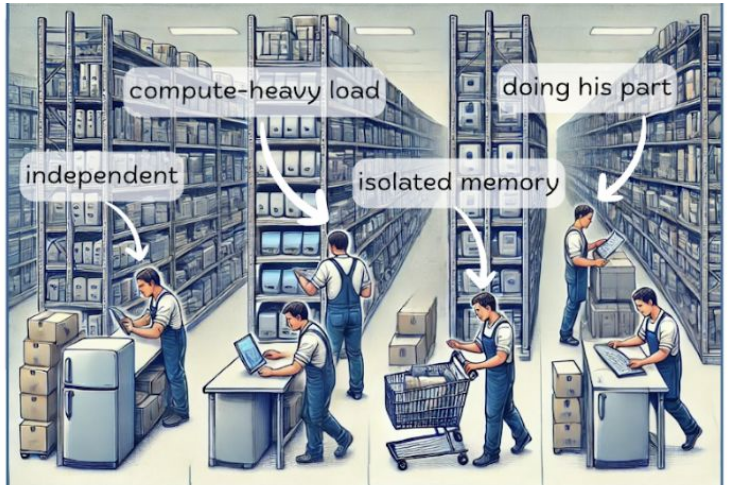


Ефективність: Поки потік чекає відповіді від БД або API, процесор перемикається на інший потік, не простоюючи даремно.

Аналогія складу: Процеси vs Потоки

Процеси

Інвентаризація (CPU-bound)



Сценарій "Інвентаризація":

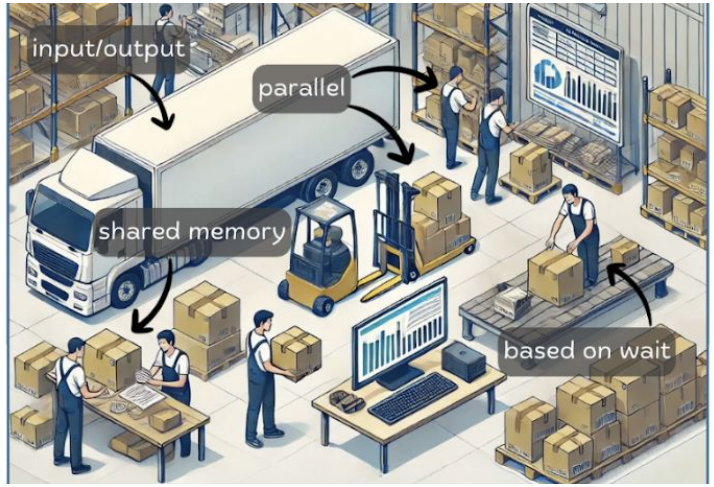
Групи працюють в окремих зонах. Одна рахує телефони, інша - холодильники. Це важкі обчислення. Робота повністю незалежна, ніхто не заважає іншому. Результат об'єднується лише в кінці.

Повна ізоляція даних (безпечно)

vs

Потоки

Відправка (I/O-bound)

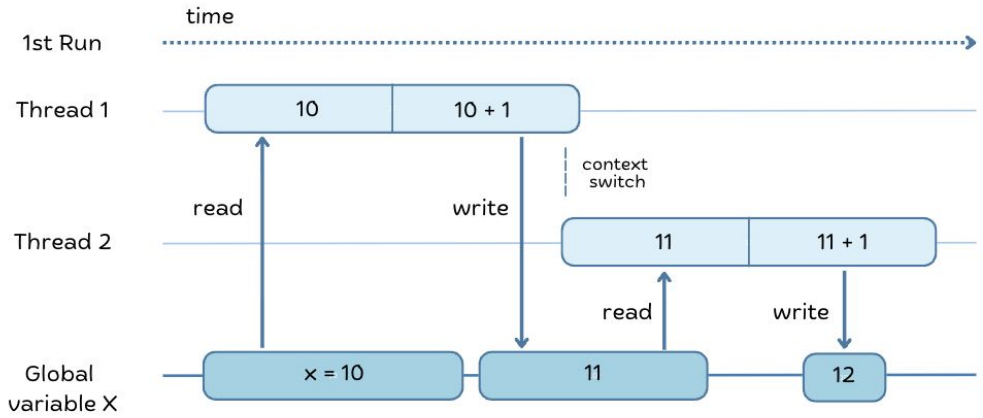


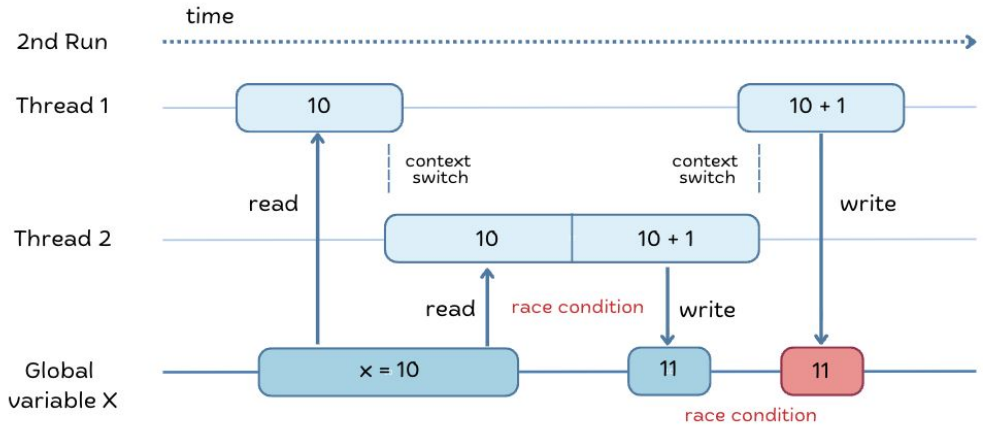
Сценарій "Відправка":

Бригада працює в одному приміщенні. Хтось вантажить, хтось реєструє. Це координація дій. Вони часто чекають (на машину, на клієнта) і постійно взаємодіють через спільний простір складу.

Спільна пам'ять (швидка комунікація)

Race Condition: Проблема



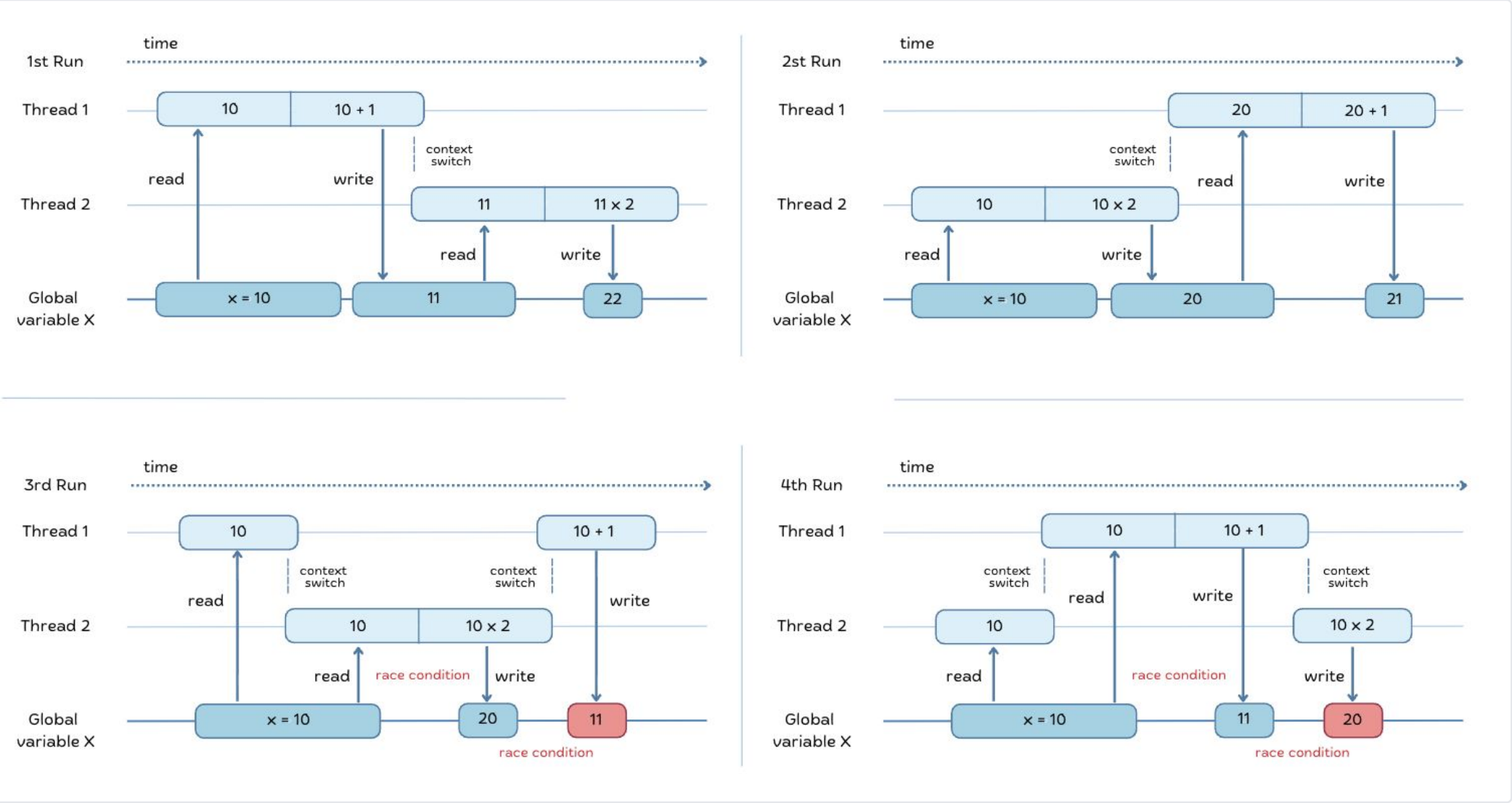


Неатомарність операцій

Проста операція x += 1 в Python не є атомарною. Вона розкладається на три окремі кроки байт-коду: читання, зміна, запис.

Втрата даних (Lost Update)
Якщо два потоки прочитають старе значення до того, як хоча б один запише нове, одна з операцій інкременту буде втрачена назавжди.

Race Condition: Візуалізація



Crashes (Segfault)

Якщо лічильник зменшиться з айвий раз, об'єкт видалиться, поки його використовують інші.

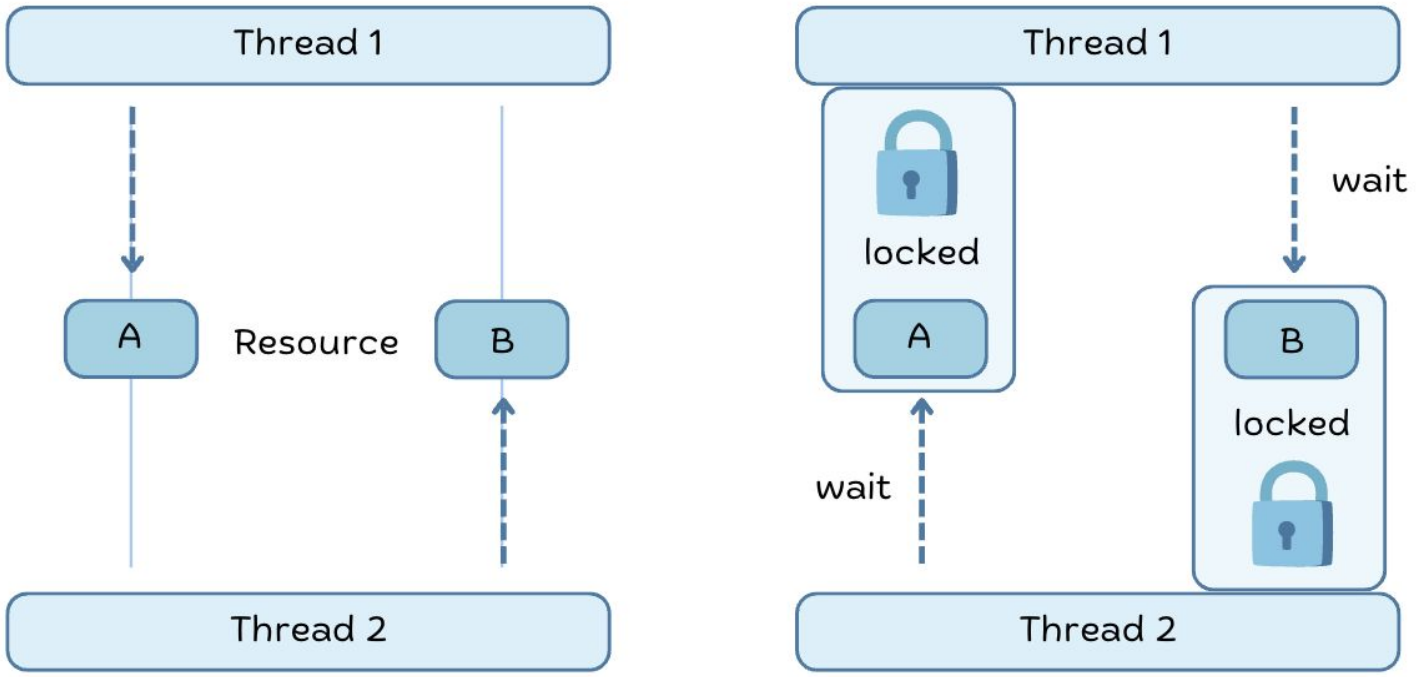
Python Bytecode (dis)

eval breakers

1) JUMP_BACKWARD
2) CALL

⚠ *Перемикання GIL release*

Lock (Mutex): Механізм роботи



Взаємне виключення

Механізм Lock гарантує, що в будь-який момент часу лише один потік може виконувати код у критичній секції. Інші потоки стають у чергу очікування.

PYTHON PATTERN

```python
import threading
lock = threading.Lock()

# Безпечний доступ
with lock:
    # Критична секція
    shared_resource.update()
    # Автоматичний release()
```

Атомарність
Операції всередині блоку стають неподільними.

Блокування
Потік спить, не споживаючи CPU.

Lock: Патерни та ризики

> Правильне використання блокувань - це баланс між безпекою даних та продуктивністю. Головне правило: тримайте Lock якомога менше.

Мінімальна критична секція

Захоплюйте замок лише навколо операцій зі спільними даними. Підготовка даних та обробка результатів мають бути ззовні.

Ніколи не блокуйте I/O

Мережеві запити або читання файлів всередині with lock: зупиняють усі інші потоки, перетворюючи програму на послідовну.

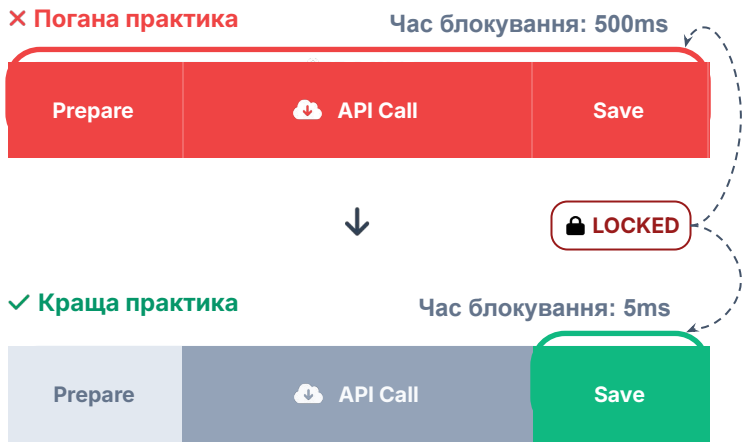
Порядок захоплення

Якщо потрібно декілька замків, завжди захоплюйте їх в одному й тому ж порядку (наприклад, A → B), щоб уникнути Deadlock.

Контекстні менеджери

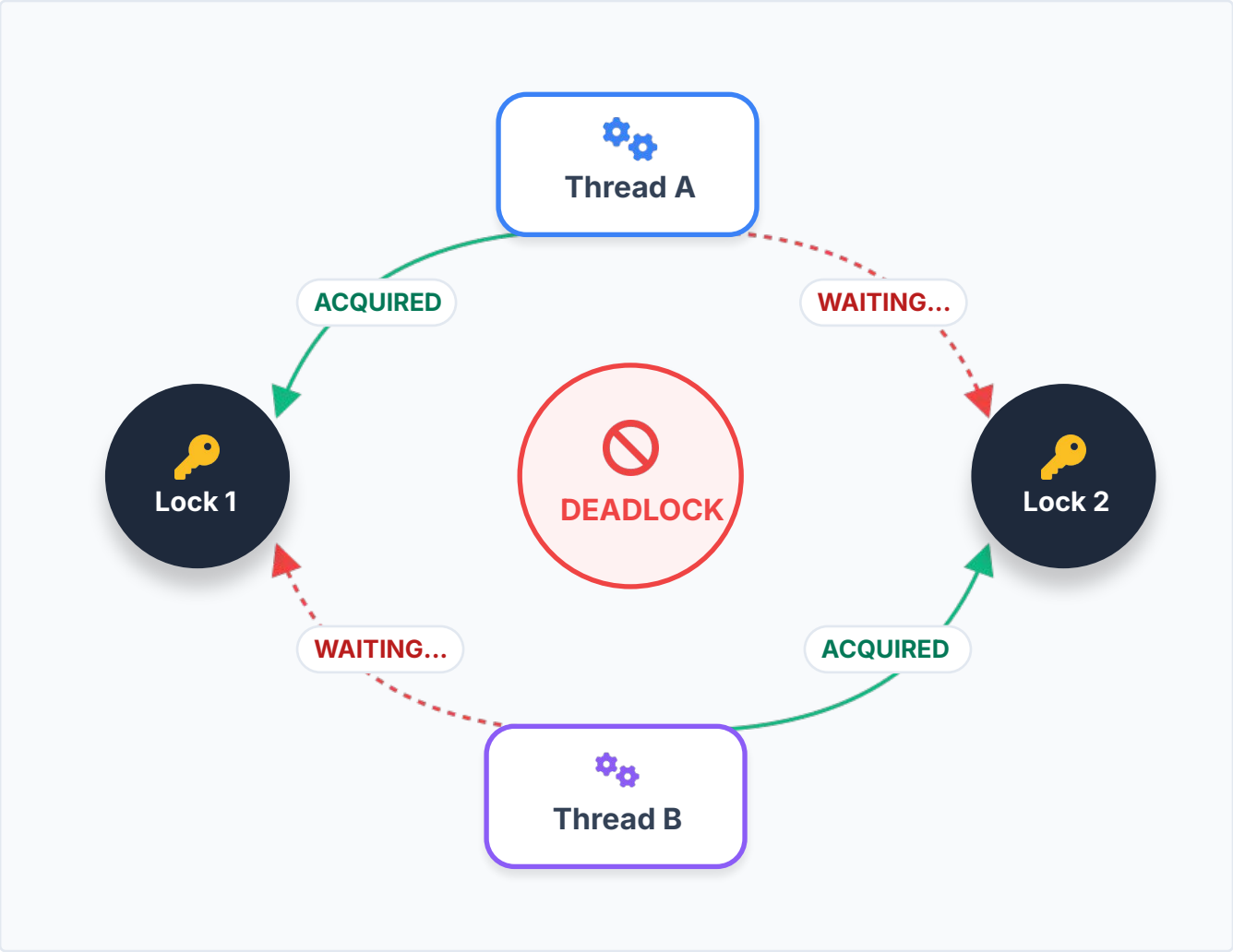
Використовуйте конструкцію with lock:. Вона гарантує звільнення замка (Release) навіть у разі виникнення помилки.

ОПТИМІЗАЦІЯ БЛОКУВАННЯ



Ризик: Якщо забути release(), всі інші потоки зупиняться назавжди (Deadlock). Використовуйте with!

Deadlock: Взаємне блокування



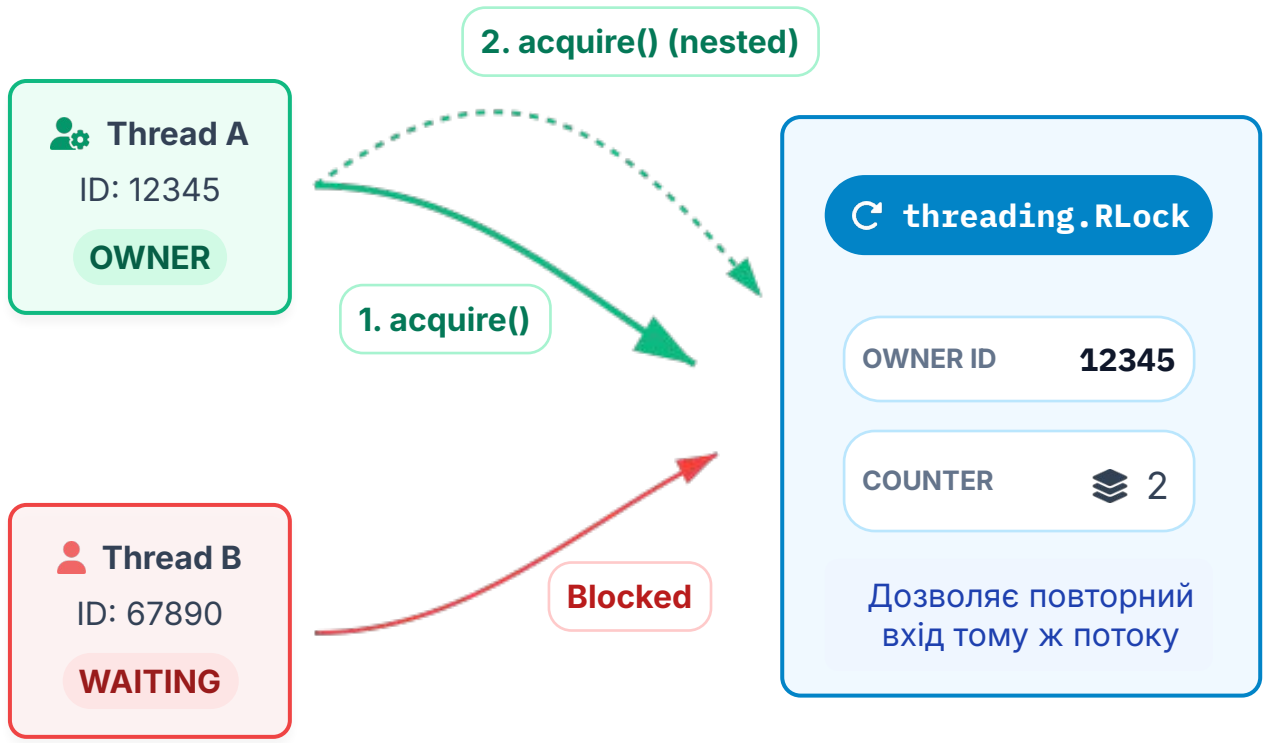
Deadlock виникає, коли група потоків чекає на ресурси, які утримують інші потоки в цій же групі, утворюючи замкнене коло.

4 УМОВИ КОФФМАНА

- **Mutual Exclusion**
  Ресурс не може використовуватись одночасно.
- **Hold and Wait**
  Потік тримає ресурс і чекає на інший.
- **No Preemption**
  Ресурс не можна відібрати силоміць.
- **Circular Wait**
  Замкнений ланцюг очікувань.

```
// Анти-паттерн вкладених локів
def thread_a():
    lock_1.acquire()
    # Context Switch here can kill us
    lock_2.acquire() # Waits for B

def thread_b():
    lock_2.acquire()
    lock_1.acquire() # Waits for A
```

RLock: Реентерабельне блокування



Що таке RLock?

Reentrant Lock дозволяє одному і тому ж потоку захоплювати блокування кілька разів без самоблокування (deadlock).

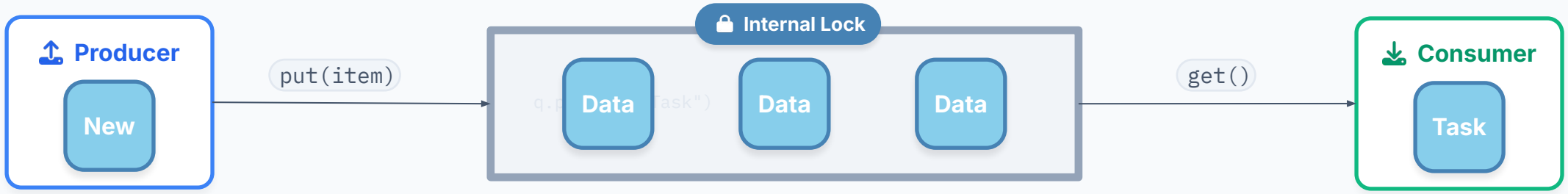
Використовується для рекурсивних функцій.
Корисний, коли методи класу викликають один одного.

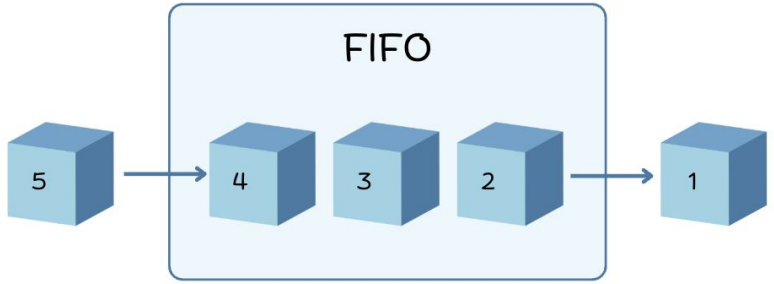
recursive_example.py

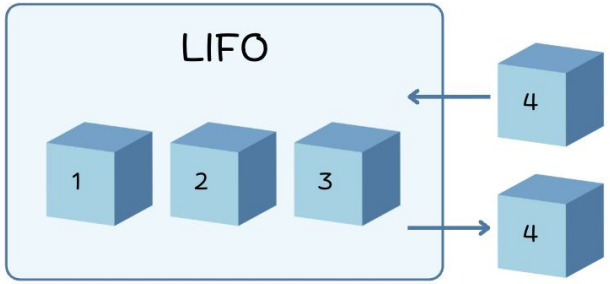
```
import threading
lock = threading.RLock()

def recursive_task(n):
   with lock: # count += 1
      if n > 0:
         recursive_task(n-1)
   # тут автоматично count -= 1
```

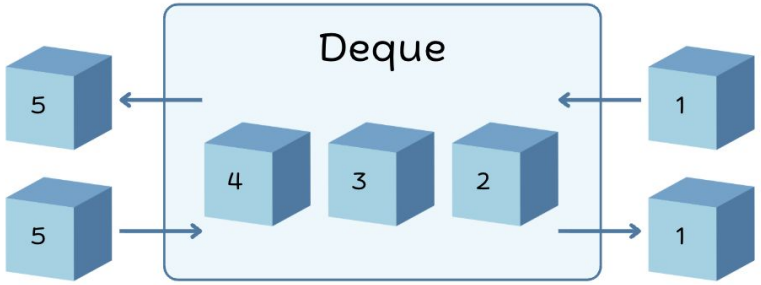
Важливо: Кількість викликів release() має дорівнювати кількості acquire(), щоб повністю звільнити ресурс.

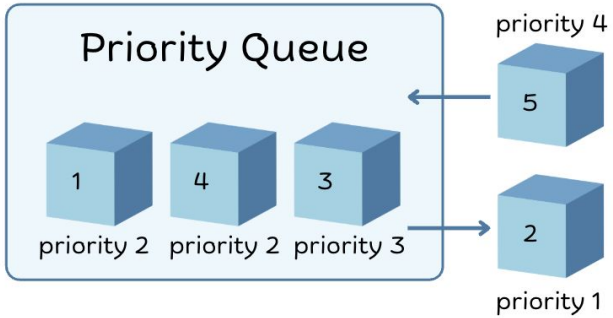
Потокобезпечні черги (Queue)









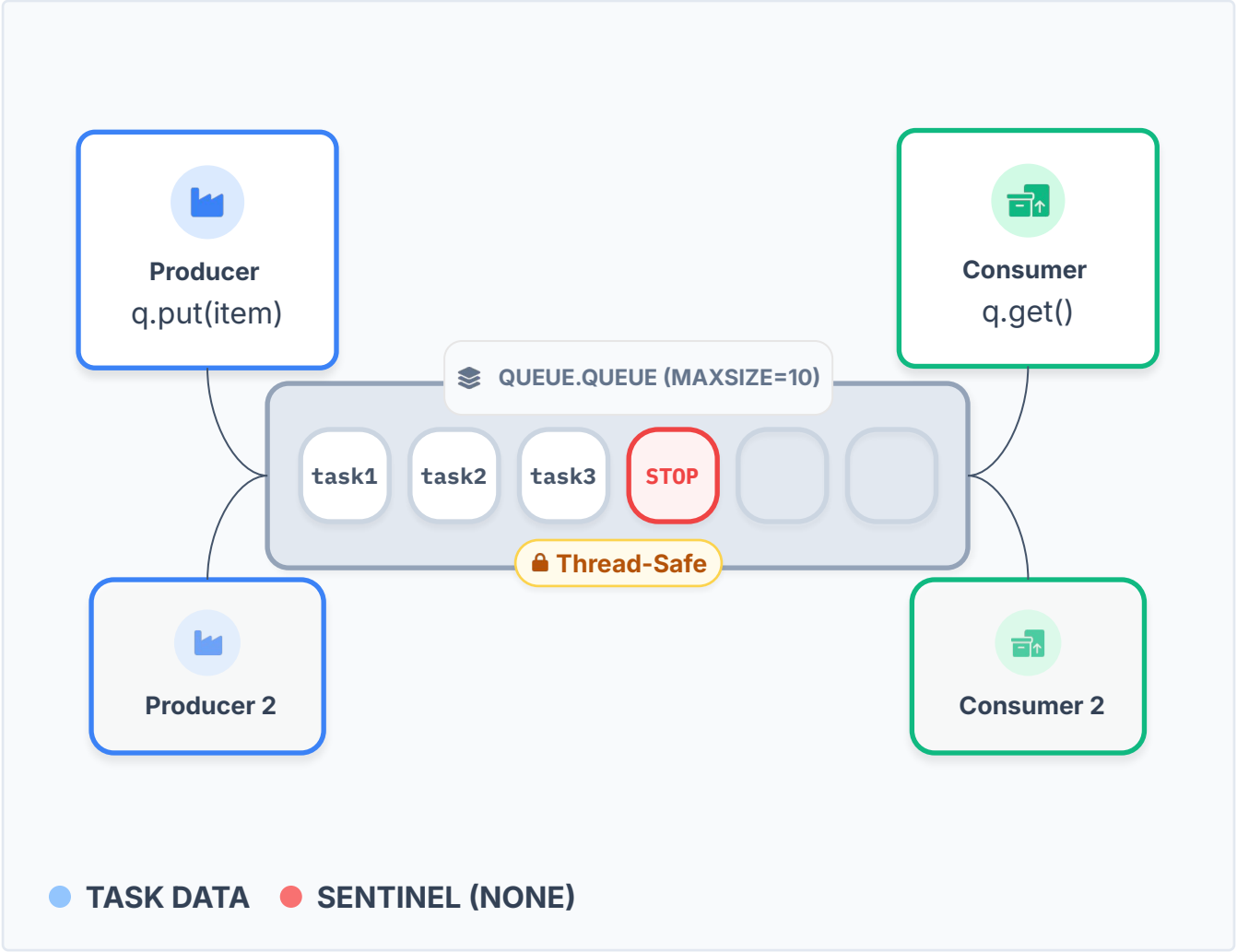


Синхронізація "з коробки"

Thread-Safe
Методи put() і get() є атомарними. Не потрібно використовувати ручні Lock.

Blocking Behavior
get() чекає дані, якщо черга пуста. put() чекає місце, якщо черга повна.

Producer-Consumer Pattern



Балансування навантаження

Використання maxsize дозволяє створити Back-pressure: Producer блокується, якщо Queue повна, не перевантажуючи пам'ять.

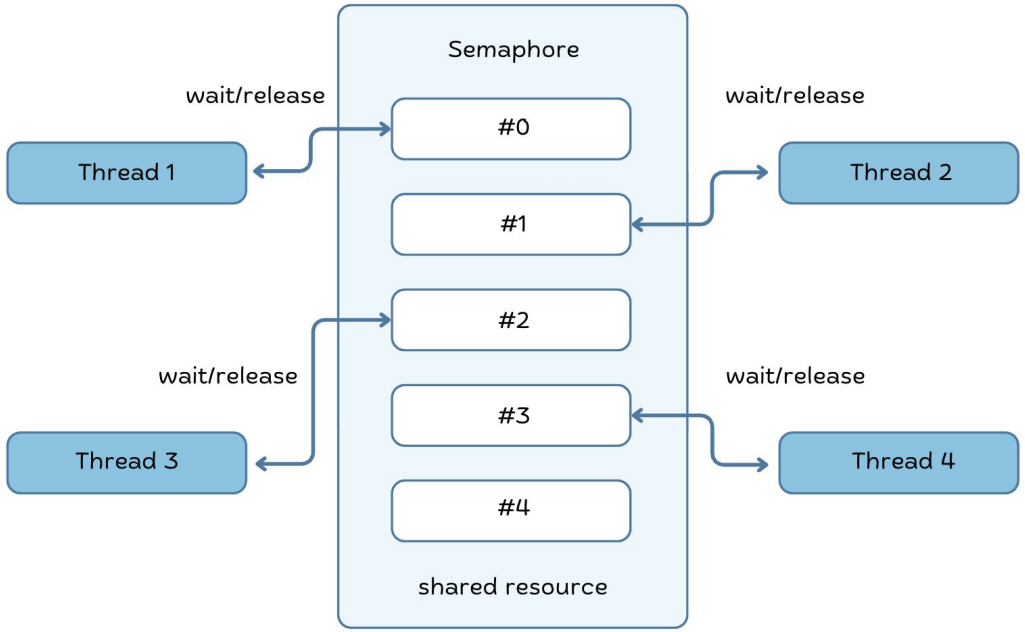
Завершення роботи (Sentinel)

Використання спеціальних маркерів (наприклад None) для сигналу Consumer'ам про завершення роботи.

pattern.py

```
# Producer Thread
queue.put(item) # Blocks if full
for _ in range(num_consumers):
    queue.put(None) # Sentinel

# Consumer Thread
while True:
    item = queue.get()
    if item is None:
        break # Graceful shutdown
    process(item)
    queue.task_done()
```

Semaphore



Що таке Семафор?

Це цілочисельна змінна, яка використовується для вирішення задач синхронізації. На відміну від м'ютекса, семафор дозволяє доступ до ресурсу N потокам одночасно.

ЗАСТОСУВАННЯ

- Пул з'єднань БД
- Producer-Consumer

Механізм лічильника (S = 2)

ACTIVE ZONE

Worker_1

AVAILABLE

1

S -= 1

WAIT QUEUE

Worker_2

Worker_3

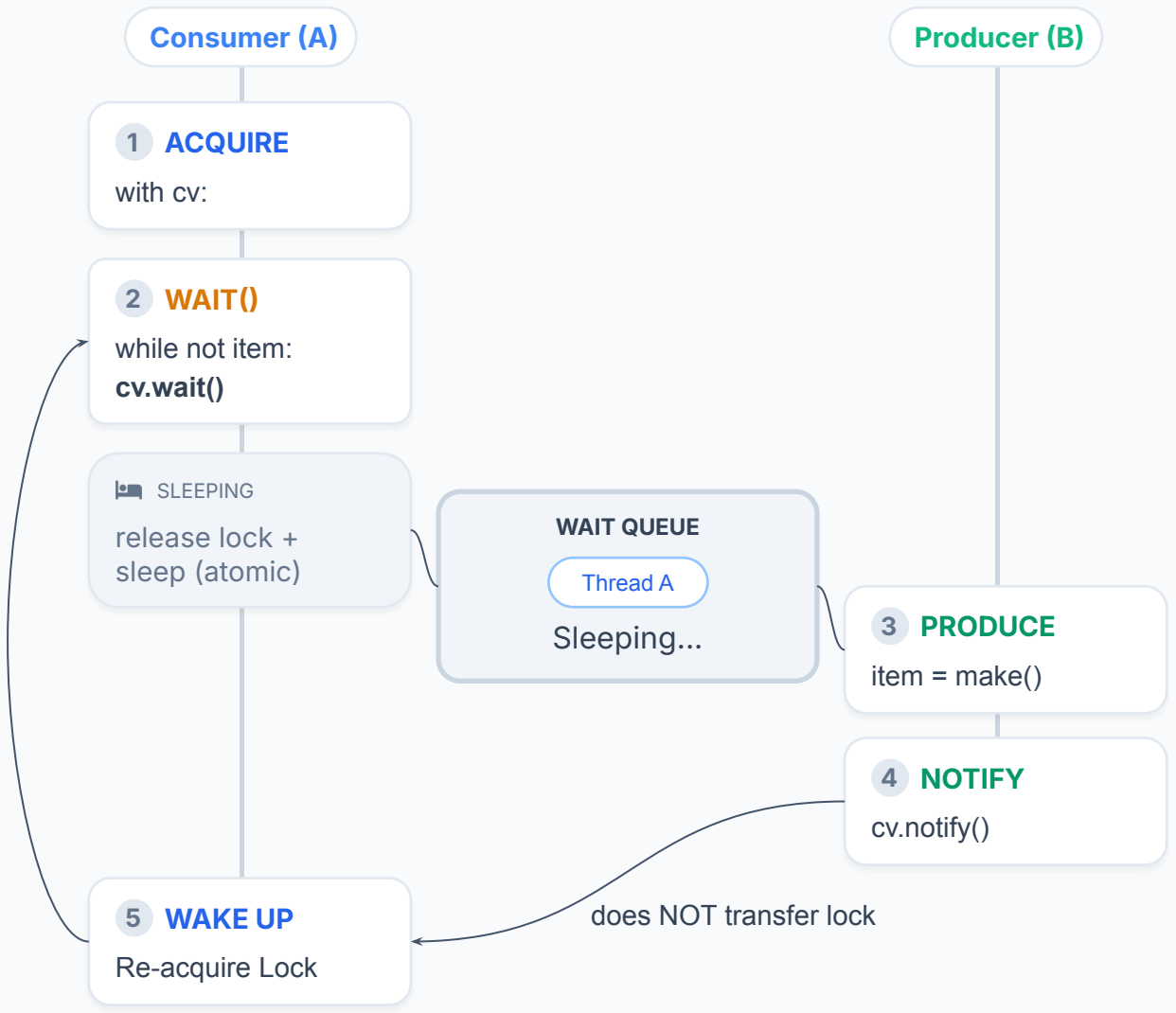
wait() if S > 0: S -= 1
else: block thread

signal() S += 1
wake up waiting thread

Semaphore

```
sem = threading.Semaphore(2)
with sem:
    do_smth()
```

Condition: Розумне очікування



Без Busy-Waiting

Замість того, щоб постійно перевіряти умову в циклі while (спалюючи CPU), потік викликає wait(). Це атомарно звільняє замок і переводить потік у режим сну.

Магія wait()

Атомарно: Звільняє м'ютекс перед сном.

Блокується: Лягає спати (Thread suspended).

При пробудженні: Знову захоплює м'ютекс.

```
Producer-Consumer Pattern

with cond_var:
   while not item_available():
      # Release lock & sleep
      cond_var.wait()
   consume_item()
```

Event: Простий сигнал

> Event - це найпростіший механізм комунікації: один потік сигналізує про подію, а інші чекають на неї.

Логіка прапора

Внутрішній прапор може бути True (set) або False (clear). Потоки, що викликають wait(), блокуються, поки прапор не стане True.

М'яка зупинка (Graceful Shutdown)

Ідеально підходить для зупинки нескінченних циклів у фонових воркерах: while not stop_event.is_set(): ...

Одноразова ініціалізація

Корисно для очікування готовності ресурсу (наприклад, з'єднання з БД), після чого всі потоки продовжують роботу.

EVENT VS CONDITION

Feature	Event	Condition
Пам'ять стану	**Так (Flag)**	Ні (Stateless)
Складність	Просто (0/1)	Гнучко (будь-яка умова)

1 Блокування (Wait)

Worker-потік викликає метод wait(). Якщо прапорець події опущений (False), виконання потоку повністю зупиняється. Це не активне очікування (busy wait), тому процесорний час не витрачається.

2 Сигнал (Set)

Головний потік (або інший потік-контролер) виконує необхідну підготовку і викликає set(). Це піднімає внутрішній прапорець події у стан True.

3 Пробудження (Resume)

ОС миттєво "розморожує" worker-потік. Він виходить зі стану wait() і продовжує виконання коду з наступного рядка.

ThreadPoolExecutor

> Високорівневий інтерфейс для асинхронного виконання завдань, що абстрагує ручне керування життєвим циклом потоків.

Перевикористання

Замість створення нового потоку для кожного завдання, пул використовує фіксований набір "воркерів", заощаджуючи ресурси системи.

Контроль навантаження

Параметр max_workers дозволяє обмежити кількість одночасних операцій, запобігаючи перевантаженню зовнішніх сервісів.

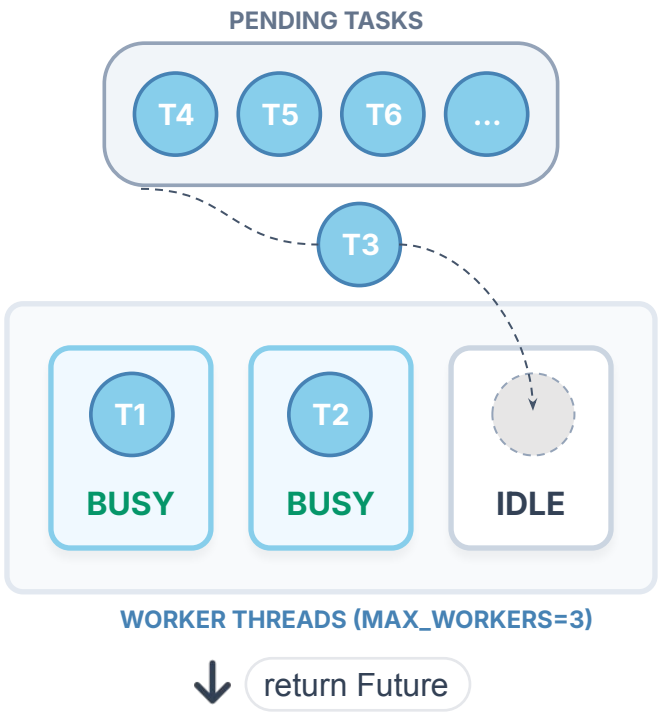
Зручний API

Методи submit() та map() повертають об'єкти Future, що спрощує отримання результатів та обробку винятків.

Context Manager

Використання with гарантує коректне завершення всіх потоків (implicit join) навіть у випадку помилок.

АРХІТЕКТУРА ПУЛУ




```
executor.submit(fn, args) # Повертає Future об'єкт миттєво
```

Порівняння примітивів синхронізації

МЕТОД / ТИП	ПРОСТОТА (USAGE)	НАКЛАДНІ ВИТРАТИ	МАСШТАБОВАНІСТЬ
Lock `Mutex`	★★★★★ Дуже просто	Low (Kernel/User)	●● Обмежена (Contention)
RLock `Reentrant`	★★★★☆ Просто	Medium (Extra checks)	●● Обмежена
Semaphore `Counter`	★★★☆☆ Середня	Low-Medium	●●● Висока (N-threads)
Condition `Signaling`	★★☆☆☆ Складно	Medium (Wait queues)	●● Середня
Queue `Communication`	★★★★★ Максимальна зручність	High (Locking + Logic)	●●● Відмінна (Decoupling)

Trade-off (Компроміс)

Queue - найбезпечніший вибір для передачі даних, попри накладні витрати. Lock - найшвидший, але найнебезпечніший (ризик Deadlock).

RECOMMENDATION

Prefer Queues

Закон Літтла

$$L = \lambda \times W$$

L — КІЛЬКІСТЬ У СИСТЕМІ

λ — ШВИДКІСТЬ ВХОДУ

W — ЧАС ПЕРЕБУВАННЯ

Якщо кожен проводить у магазині 30 хвилин, а нові відвідувачі заходять кожні 6 хвилин,то всередині постійно знаходиться 5 людей.

Приклад: Магазин

Швидкість надходження (λ)	10 люд/год
Час перебування (W)	0.5 год
Середня кількість (L)	**5 людей**

Як це стосується IT?

Закон Літтла допомагає зрозуміти, скільки одночасних потоків (або запитів в черзі) потрібно підтримувати системі.

```
λ = 10                    // Розрахунок
W = 0.5
L = 10 * 0.5 = 5
```

Оптимальна кількість потоків

ФОРМУЛА РОЗРАХУНКУ

$$N_{opt} \approx (T_{cpu} + T_{wait}) / T_{cpu}$$

Мета:
Максимізувати завантаження CPU, коли потоки очікують (I/O)

Змінні та Теорія

N_{opt} — **Кількість потоків**
Оптимальне число для запуску

T_{cpu} — **Час обчислень**
Корисна робота процесора

T_{wait} — **Час очікування**
I/O, мережа, відповідь БД

"Ми додаємо потоки, щоб заповнити паузи в роботі CPU корисними діями інших потоків."

Практичний приклад

Case Study

ОБРОБКА ДАНИХ Tcpu
50 ms = 0.05 s

ЗАПИТ ДО API Twait
450 ms = 0.45 s

Загальний час операції (500 ms)

10% 90%

РОЗРАХУНОК:

```
N = (0.05 + 0.45) / 0.05
```

Результат: **10** потоків

Кращі практики та Анти-патерни

Багатопотоковість вимагає дисципліни. Ось ключові правила для написання надійного та безпечного коду.

Queue замість Global State

Уникайте спільних змінних. Використовуйте потокобезпечні черги (Queue) для передачі даних між потоками.

Безпека виконання

Завжди використовуйте timeout при блокуваннях та обробляйте винятки всередині кожного потоку окремо.

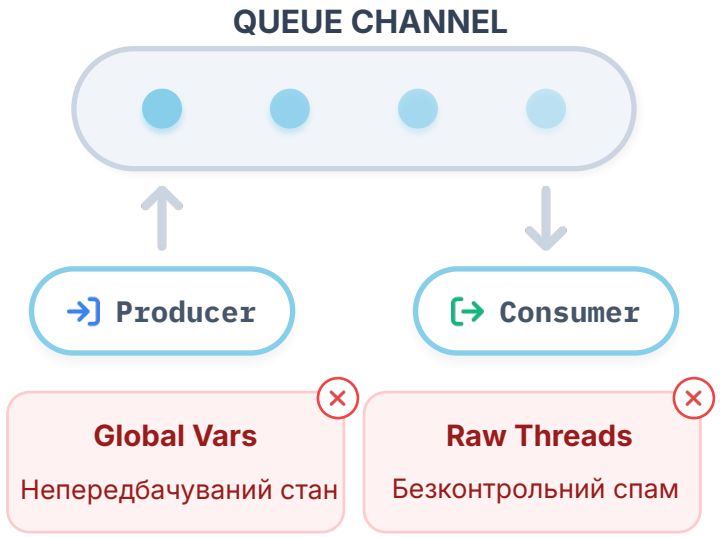
Sleep-синхронізація

Ніколи не використовуйте time.sleep() для очікування подій. Це ненадійно і призводить до race conditions.

Надмірні потоки

Створення тисяч потоків "вбиває" продуктивність перемиканням контексту. Використовуйте пули (Pools).

АРХІТЕКТУРА



Золоте правило: Якщо потоки не спілкуються через черги, ви, ймовірно, створюєте майбутній deadlock.

Ключові тези та рекомендації

01

Потоки для I/O задач

Використовуйте threading для мережевих запитів, роботи з БД та диском. GIL звільняється під час системних викликів, дозволяючи справжню конкурентність без блокування CPU.

02

Високорівневі примітиви

Уникайте ручного керування низькорівневими Lock(). Надавайте перевагу Event, Condition та Semaphore для більш чистого, безпечного та зрозумілого коду синхронізації.

03

Спочатку профілювання

Не оптимізуйте наосліп. Використовуйте інструменти типу py-spy або viztracer, щоб виявити реальні блокування GIL та вузькі місця перед рефакторингом.

04

Комунікація через черги

Уникайте спільного глобального стану. Використовуйте queue.Queue для безпечного обміну даними між потоками (Producer-Consumer паттерн).

> *"Багатопотоковість - це інструмент, а не магія. Розуміння механізму GIL перетворює його з ворога на передбачуваного партнера."*

PYTHON CONCURRENCY

Що почитати?

- Python Threading: The Complete Guide
- Threading
- Python Thread Safety
- Manage Concurrent Operations Within a Process
- CPython - threading.py